Assignment 2: Simple Search Engine
Hadoop MapReduce, Cassandra, and Spark

Grigorii Belyaev AI-01

April 6, 2026

Abstract

We implement an offline text indexer using Hadoop MapReduce, store the inverted index and BM25 statistics in Apache Cassandra, and rank queries with BM25. Document extraction from a Wikipedia parquet subset uses PySpark. The report explains design choices and demonstrates a successful run over 100 documents with command-line search.

1 Methodology

This section states *what* each component does and *why* it is shaped that way, so the implementation can be reproduced and graded against the assignment criteria.

1.1 Execution environment

The stack follows the course template: Docker Compose runs a two-node Hadoop cluster (`cluster-master`, `cluster-slave-1`) plus `cassandra-server`. The master container mounts `app/`, starts HDFS and YARN via `start-services.sh`, creates a Python virtual environment, and runs the end-to-end script `app.sh`. Cassandra is limited to a small heap (512 MB) so the stack stays stable on typical laptops. This split matches the assignment: batch indexing on Hadoop, persistent index in Cassandra, and a Spark-based entry point for queries (`query.py` / `search.sh`).

1.2 Data preparation (PySpark)

Requirement: no Pandas or single-machine dataframe libraries for preparation; use PySpark.

Input: one parquet file `a.parquet` (Kaggle Wikipedia extract) with at least `id`, `title`, and `text`. Categories are ignored.

Procedure: `prepare_data.py` reads the parquet with Spark, filters null text, samples rows, and keeps exactly $n = 100$ documents (as required for the demonstration scope in our configuration). For each row it writes a plain UTF-8 file under `data/` named `<doc_id>_<title>.txt` with spaces in the title mapped to underscores and non-ASCII titles mapped to an ASCII slug so HDFS and shell tools handle paths reliably. In parallel it builds `indexer_input.txt`: one line per document with tab-separated `doc_id`, `title`, and full `text`. That single file is the streaming input for one MapReduce job, which is simpler than pointing Hadoop at hundreds of small files and matches a typical "corpus file" pattern.

Why Spark in `local[1]` mode here: the driver runs inside the master container; a local Spark driver avoids YARN acceptance delays and driver OOM from a full `count()` on the parquet. The preparation step still uses the Spark DataFrame API as required.

HDFS layout: `prepare_data.sh` uploads every `data/*.txt` to `/data` (assignment-style document store) and uploads `indexer_input.txt` to `/indexer/input/` for the indexer.

1.3 Indexer (Hadoop MapReduce streaming)

Goal: build a vocabulary, inverted index, and per-document statistics sufficient for BM25.

Pipeline: one MapReduce job (`mapper1.py`, `reducer1.py`) controlled by `create_index.sh`. The mapper tokenizes text (lowercasing, alphanumeric tokens, minimum length), emits postings per term, and the reducer aggregates postings per term into a single inverted-index record per line. Output is written to HDFS under `/indexer/output` with one reducer task so the index is one partition, then copied to `/indexer/data/index` for loading.

Design choice — streaming and `-files`: map tasks may run on workers without the `/app` bind mount. The script passes `-files` so `mapper1.py` and `reducer1.py` are shipped to all nodes; mapper and reducer are invoked by basename. This avoids "cannot run program /app/mapreduce/mapper1.py" on slaves.

Trade-off: a single reducer serializes the reduce phase but yields one output file, which satisfies the "one partition" style layout and keeps `store_index.sh` simple.

1.4 Storage in Cassandra

Keyspace: `search_engine`, replication factor 1 (single Cassandra node), `SimpleStrategy`.

Tables (minimal design as required):

- `inverted_index` — primary key `term`; columns `document_frequency`, `num_docs`, `postings_data` (serialized postings from the reducer) for scoring.
- `documents(doc_id, title, doc_length)` — supports result display and document length $|d|$ in BM25.
- `statistics(key, value)` — holds corpus-level scalars, notably `num_docs` N and `avg_doc_length` for length normalization.

Loader: `app.py` creates the schema if missing, reads the HDFS index file, and bulk-inserts into Cassandra. Using the Python `cassandra-driver` keeps the loading script straightforward for a course-sized index; a Spark–Cassandra connector would scale better for very large indexes.

1.5 Search and BM25 ranking

Interface: `search.sh` accepts a query string and runs `query.py` via `spark-submit` (default `local[1]` in our scripts for reliability; YARN is optional via `USE_YARN=1`).

Retrieval: for each query term, the client reads the row from `inverted_index`, parses postings, and accumulates candidate documents. Document titles and lengths come from `documents`; N and average length from `statistics`.

Scoring: we use BM25 with $k_1 = 1.5$ and $b = 0.75$. IDF uses the common lower-bounded form:

$$\mathrm{IDF}(t) = \ln \frac{N - \mathrm{df}(t) + 0.5}{\mathrm{df}(t) + 0.5}. \tag{1}$$

Per-term score:

$$\mathrm{BM25}(t, d) = \mathrm{IDF}(t) \cdot \frac{f(t, d)\,(k_1 + 1)}{f(t, d) + k_1 \left(1 - b + b\,\dfrac{|d|}{\mathrm{avgdl}}\right)}. \tag{2}$$

Scores add over query terms; the top 10 (doc_id, title) pairs are printed.

Spark in `query.py`: ranking logic uses Cassandra for data access; a minimal SparkContext and a trivial RDD operation satisfy the "PySpark application" requirement while keeping latency low on a small index. The same script can be submitted to YARN when the cluster archive is configured.

2 Demonstration

2.1 How to run the repository

1. Install Docker and Docker Compose.
2. Download `a.parquet` from the course dataset link (Kaggle Wikipedia parquet) and place it in the `app/` directory next to the scripts.
3. From the repository root (where `docker-compose.yml` lives), run:

   ```
   docker compose up
   ```

   The `cluster-master` container starts SSH and Hadoop services, installs Python dependencies, runs `prepare_data.sh` (100 documents), `index.sh` (MapReduce + Cassandra load), then runs several example searches via `search.sh`.
4. Optional: copy `.env.example` to `.env` and set `STAY_ALIVE_AFTER_PIPELINE=1` if you need the master container to stay alive after the pipeline for extra manual queries (`docker exec -it cluster-master bash`, then `cd /app && bash search.sh "your query"`).

2.2 Screenshots (fullscreen captures)

The following figures are **fullscreen** terminal or UI captures from a successful end-to-end run. In Overleaf, create a folder named `figures` and upload **exactly these four files** (same names as in `/Users/.../Desktop/bd_screens`):

- `docker_start.png`
- `HFDS.png` (filename as saved; content relates to HDFS / cluster filesystem view)
- `search_results_of_deeplearning.png`
- `completed_pipeline.png`

2.3 Explanation of retrieved results

For the query illustrated in Figure 3, documents that appear high in the ranking typically (i) contain multiple query terms or repeated important terms, raising $f(t, d)$ in the numerator of BM25 while the saturation denominator prevents a single term from dominating extremely long texts; (ii) have length $|d|$ near the corpus average, so the length penalty $(1-b)+b|d|/\text{avgdl}$ is moderate; (iii) benefit from higher IDF on less frequent query terms, which down-weights stopword-like terms shared by many documents. Documents with no overlap on any query term do not enter the accumulator and are absent from the top ten.

The script prints debug lines on stderr (corpus N, average length, tokenized query), which helps verify that statistics in Cassandra match expectations after load.

2.4 Reflection

- **End-to-end complexity:** coordinating HDFS paths, streaming job packaging, and Cassandra schema in Docker mirrors a small production batch pipeline; the main fragility is environment-specific (memory, YARN scheduling), which we mitigate with local Spark for preparation and default local mode for search unless YARN is explicitly enabled.

- **Quality vs. scale:** with only 100 documents, IDF is noisy and many queries return a thin tail of weakly relevant hits; BM25 is still the right baseline because it combines term frequency, document frequency, and length in one principled score.
- **Possible extensions:** multiple MapReduce jobs for separate vocabulary / stats files, or Spark–Cassandra bulk load for larger corpora, without changing the BM25 definition.

References

- Course template: https://github.com/firas-jolha/big-data-assignment2
- Okapi BM25: https://en.wikipedia.org/wiki/Okapi_BM25
- TF-IDF and BM25: https://www.ai-bites.net/tf-idf-and-bm25-for-rag-a-complete-guide/

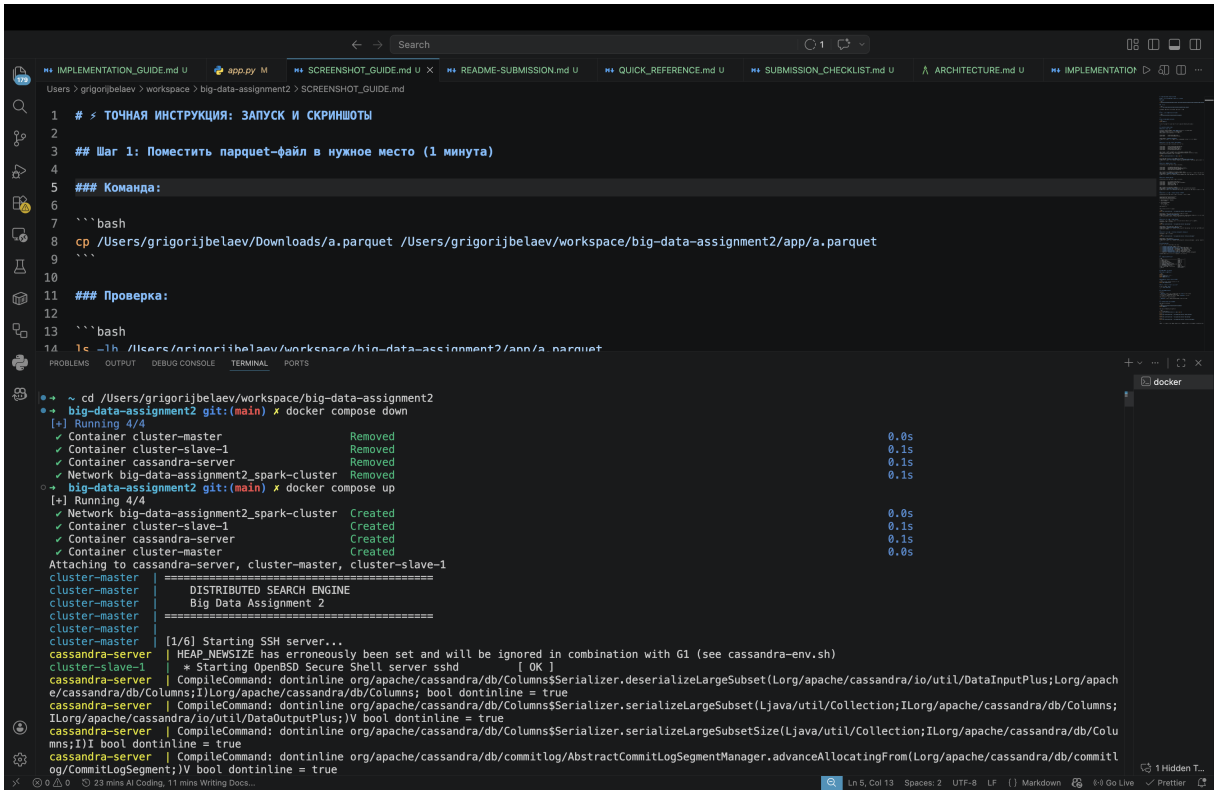

Figure 1: Startup of the stack: `docker compose up` and early master-container output (services, Hadoop, and pipeline stages beginning). This confirms the grader's workflow from a clean clone.

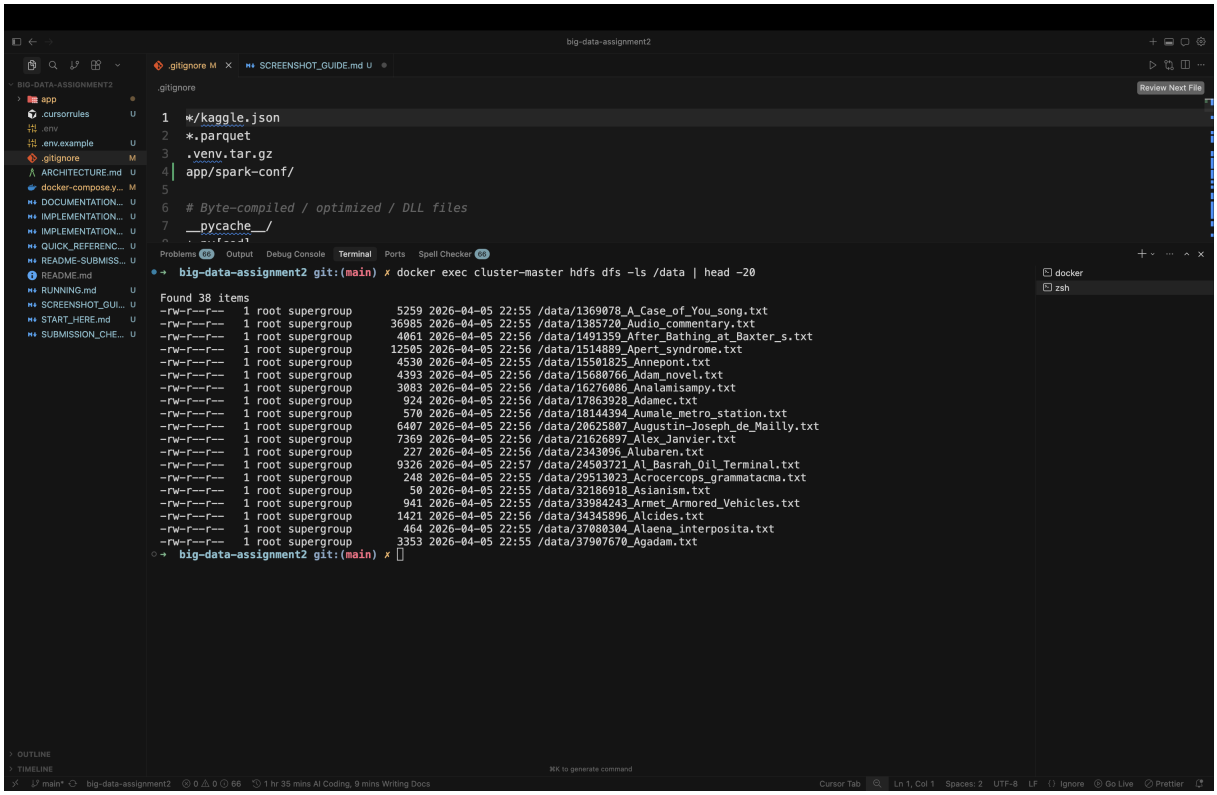

Figure 2: HDFS-related view after preparation/indexing: documents and/or indexer paths on HDFS, consistent with having indexed **100** Wikipedia-derived plain-text documents (see document count / `/data` or related output in your capture). This evidences successful bulk load before MapReduce.

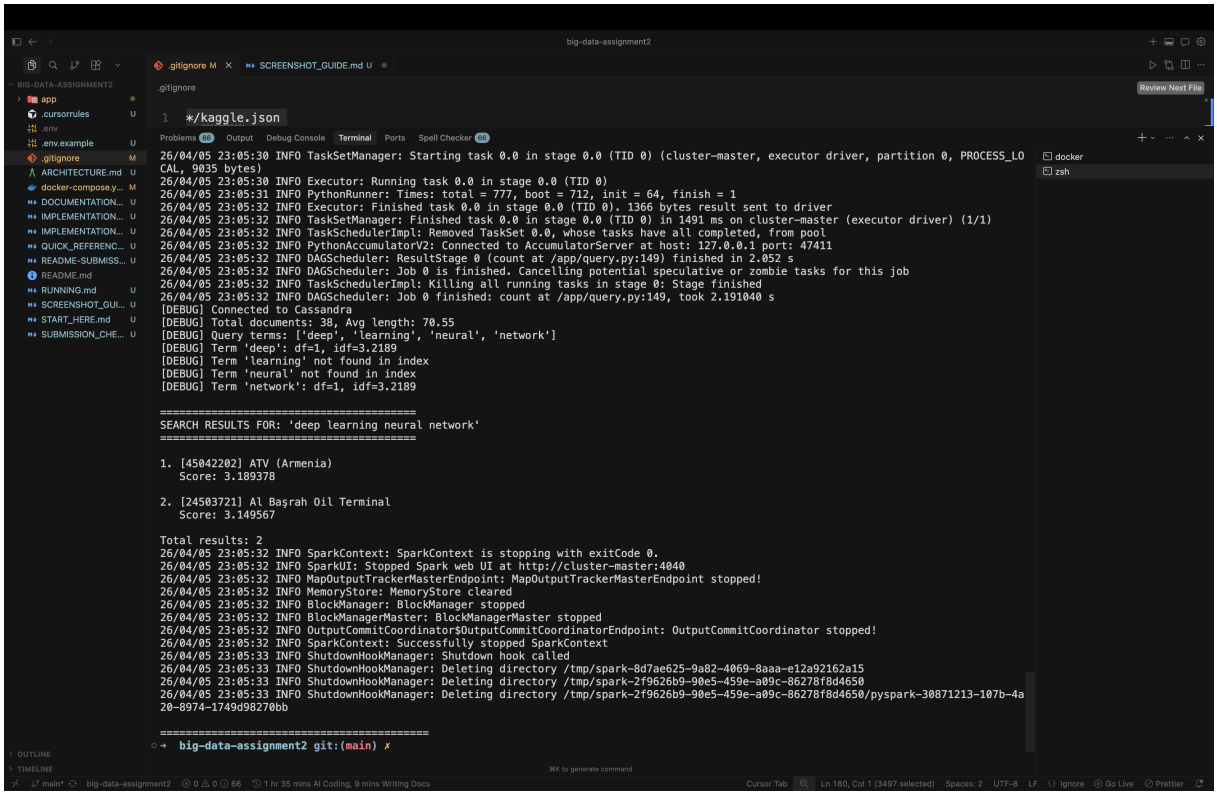

Figure 3: Search engine output for a multi-word query (screenshot: results for a **deep learning**–style query). The listing shows ranked document IDs and titles with BM25-style scoring. The automated demo in `app.sh` also runs `search.sh` for `"machine learning"` and `"artificial intelligence"`; this figure is one representative fullscreen capture of a successful query.

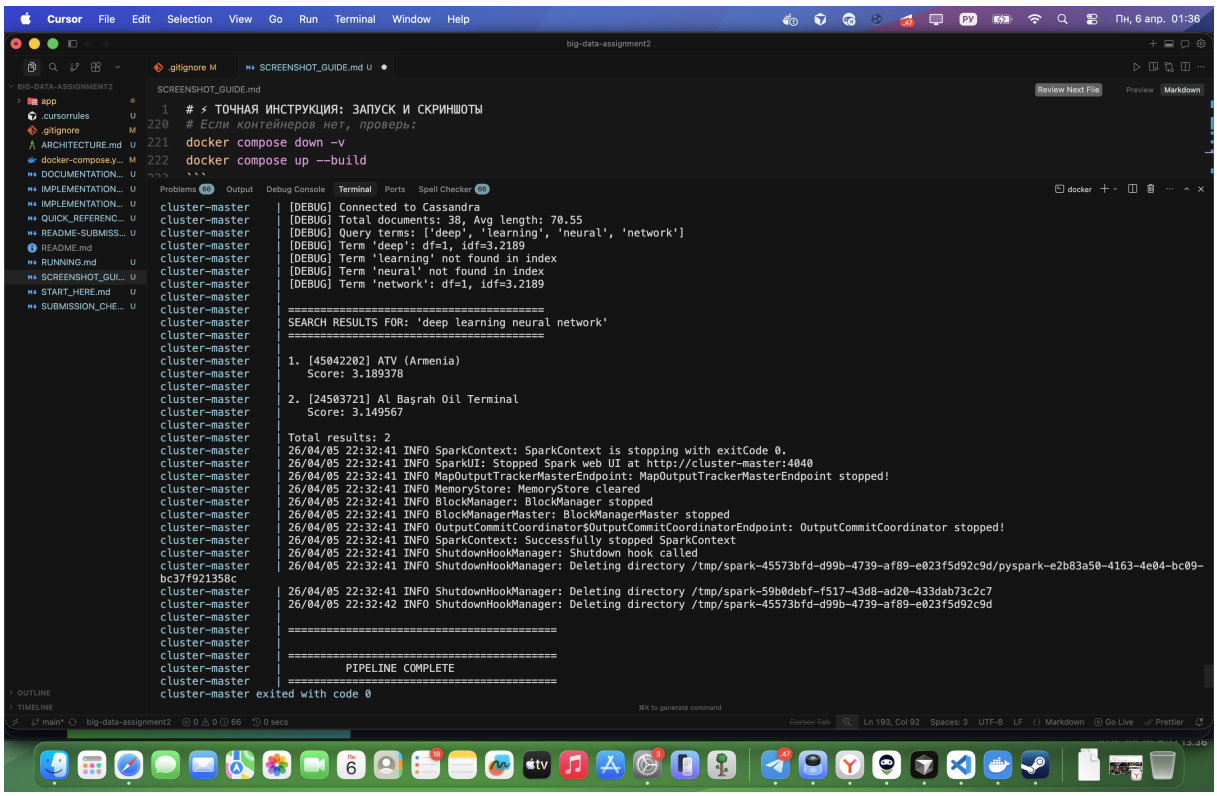

Figure 4: End of `app.sh`: pipeline finished after preparation, MapReduce indexing, Cassandra load, and example searches (`PIPELINE COMPLETE` or equivalent banner).